

Terraform Cloud

DevOps Cheat Sheet · GCP Engineer Edition

CORE CLI COMMANDS

<code>terraform init</code>	Download providers & set up backend	<i>Run first in every new project</i>
<code>terraform plan</code>	Preview what will change (dry run)	<i>Always do this before apply</i>
<code>terraform apply</code>	Create / update real infrastructure	<i>Adds -auto-approve to skip prompt</i>
<code>terraform destroy</code>	Delete all resources in state	<i>Destructive — use with caution</i>
<code>terraform fmt</code>	Auto-format .tf files to style standard	
<code>terraform validate</code>	Check syntax without connecting to cloud	
<code>terraform output</code>	Print output values from state	<i>e.g. IP address, bucket name</i>
<code>terraform show</code>	Read the current state file	
<code>terraform state ls</code>	List all resources tracked in state	
<code>terraform import</code>	Bring existing resource under TF control	<i>terraform import res.id cloud_id</i>
<code>terraform workspace new NAME</code>	Create a new workspace	<i>Separate state per environment</i>

HCL BUILDING BLOCKS

<code>resource</code>	Declares a real infrastructure object	<i>resource "google_..." "name" {}</i>
<code>data</code>	Read existing resource not managed by TF	<i>data "google_..." "name" {}</i>
<code>variable</code>	Input parameter for the module/config	<i>Passed via .tfvars or CLI</i>
<code>output</code>	Export a value after apply	<i>Good for IPs, URLs, IDs</i>
<code>locals</code>	Reusable computed values within a file	<i>locals { env = "prod" }</i>
<code>module</code>	Call a reusable block of Terraform code	<i>module "vpc" { source = "../vpc" }</i>
<code>provider</code>	Configure a cloud provider (GCP, AWS...)	<i>provider "google" { project = ... }</i>
<code>backend</code>	Where to store the state file	<i>backend "gcs" { bucket = ... }</i>
<code>terraform {}</code>	Root block: required_providers, backend	<i>Always present in main.tf</i>

COMMON GCP RESOURCES (google provider)

<code>google_compute_instance</code>	A virtual machine (GCE)	<i>machine_type, boot_disk, network</i>
<code>google_compute_network</code>	VPC network	
<code>google_compute_subnetwork</code>	Subnet inside a VPC	
<code>google_storage_bucket</code>	Cloud Storage bucket	
<code>google_sql_database_instance</code>	Cloud SQL instance	
<code>google_container_cluster</code>	GKE Kubernetes cluster	
<code>google_bigquery_dataset</code>	BigQuery dataset	
<code>google_project_iam_member</code>	Grant IAM role to a user/SA	<i>role, member fields required</i>
<code>google_service_account</code>	Create a GCP service account	
<code>google_pubsub_topic</code>	Pub/Sub topic	

VARIABLES, STATE & WORKSPACES

<code>terraform.tfvars</code>	Default variable values file	<i>Auto-loaded by terraform</i>
-------------------------------	------------------------------	---------------------------------

<code>*.auto.tfvars</code>	Also auto-loaded variable files	
<code>-var="key=val"</code>	Pass a variable via CLI	<i>terraform apply -var="env=prod"</i>
<code>TF_VAR_name</code>	Set variable via environment variable	<i>export TF_VAR_region=us-central1</i>
<code>terraform.tfstate</code>	Local state file — tracks real resources	<i>Never edit manually</i>
Remote state (GCS)	Store state in Cloud Storage bucket	<i>Enables team collaboration</i>
State locking	Prevents two people applying at once	<i>Automatic with GCS backend</i>
<code>terraform.workspace</code>	Current workspace name as a value	<i>Use in resource names for isolation</i>

MINIMAL GCP EXAMPLE

```
terraform { required_providers { google = { source = "hashicorp/google", version = "~> 5.0" } } backend "gcs" { bucket = "my-tf-state", prefix = "prod" } } provider "google" { project = var.project_id region = "us-central1" } resource "google_storage_bucket" "assets" { name = "my-app-assets-${var.env}" location = "US" } variable "project_id" {} variable "env" { default = "dev" } output "bucket_url" { value = google_storage_bucket.assets.url }
```

Tip: Always run terraform plan before terraform apply — review every change carefully.

developer.hashicorp.com/terraform